Getting Groundwork into Go High Level

Requested at the 20 August meeting: a plan and a time estimate for giving the team contact visibility over both contractors and users.

Two phases. The first buys visibility quickly using what is already built. The second is the real integration, and it is what unlocks anything involving pipelines, tags or two-way conversation.

What is connected today is less than it looks

Groundwork does not use the Go High Level API. It posts a message into a single webhook whenever a contractor applies — and that is the whole integration.

Contractors only. Not one homeowner or client has ever reached the CRM. That is the gap you described in the meeting, and it is total rather than partial.

Nothing can be changed after it is sent. We never record the CRM's own id for a contact, so we cannot go back and update one, tag it, or move it along a pipeline. Every message is fire-and-forget. This is the single fact that makes Phase 2 necessary rather than optional — no amount of extra webhooks gets around it.

Failures are invisible. If a push fails, the error is discarded in the browser. There is no retry, no queue, and no screen anywhere that shows what did not make it through.

PHASE 1 · START HERE

Visibility, without rebuilding anything

Each of these copies the pattern that already works for contractor applications. No new plumbing, no database changes, nothing that has to be undone later.

WHAT YOU GET	WHY IT MATTERS	DAYS
User & homeowner contacts	Every person who signs up reaches the CRM with their name, email, country and language. Today none of them do — GHL only ever hears about contractors.	1.0
Application decisions	When you accept or reject a contractor, the CRM finds out. Right now it still shows them as a fresh application forever.	0.5
Subscription changes	Upgrades, cancellations and failed payments reach the CRM, so you can see who is paying without opening Stripe.	0.5
Project created	A signup who actually starts a build looks different from one who never does. This is the event that separates them.	0.5
Failure visibility	Today a CRM push can fail and nobody finds out — the browser throws the error away. Add a resend button and a count of what has not synced.	1.0
Phase 1		**3.5 days**

Half of this is not engineering. The code delivers contacts and events into GHL; it does not decide what happens to them. Until someone builds the receiving workflows in GHL, Phase 1 produces nothing you can see. Worth starting that in parallel rather than after.

PHASE 2 · THE REAL INTEGRATION

A CRM that can actually be driven

Everything here depends on the second item. Until we can address a contact by its CRM id, the rest is impossible rather than merely unbuilt.

WHAT YOU GET	WHY IT MATTERS	DAYS
Authenticated connection	Replace the anonymous webhook with a real, authenticated connection to GHL — a Private Integration Token, which needs no login-refresh machinery to maintain.	1.0
Remember who is who	Store the CRM's own id for each contact. Without this nothing can ever be updated after it is created, which is the ceiling on everything below.	1.0
Update instead of duplicate	Match on email so a person who signs up, then applies, then subscribes is one contact with a history — not three.	1.0
Tags and pipeline stages	Move people through your sales pipeline automatically as they progress, instead of someone dragging cards by hand.	1.5
Listen back from GHL	Let GHL tell us things too — a booked appointment, a reply, an unsubscribe. Everything today is one-way.	1.5
Retry what fails	A CRM outage currently loses the event permanently. Queue it and try again.	1.0
Phase 2		**7 days**

What the estimate assumes

Roughly 10.5 engineer-days in total — about three working weeks of undivided attention, and longer in practice.

These are build-and-test days for one engineer, and they assume the GHL account, its pipeline and its workflows already exist to build against. They do **not** include the GHL-side configuration listed below, discovery time on the GHL API, or the admin dashboard work this competes with. As discussed, taking Phase 1 next week means the dashboard moves.

Suggested order: **Phase 1 first, then stop and use it.** A month of real contacts flowing in will tell you which Phase 2 items you actually want, and that is cheaper than deciding now.

Not included — this is your side

Work that happens inside Go High Level, not in the product.

- Building the workflows in GHL that decide what happens when a contact arrives.
- Defining the pipeline and its stages.
- Writing the email and SMS sequences the CRM will send.
- Deciding who on the team owns replying to what.

Decisions needed before anyone starts

Four answers. The first two shape what gets built; the last two are approvals.

1. **Which pipeline stages exist?**

   Phase 2 moves people between stages automatically. We need the list you actually use in GHL before anything can be wired to it.

2. **Which events matter commercially?**

   The Phase 1 list is what is cheap to send. You should cut anything that will not change how someone is followed up — every event added is a workflow someone has to maintain.

3. **The waitlist path — revive or delete?**

   A complete waitlist signup flow, including its own CRM push and welcome email, is still deployed but unreachable since the community feature was removed. It is either revived deliberately or deleted; leaving it is the worst of the three.

4. **Homeowner data going to a subprocessor**

   Contractors already flow to GHL and it is disclosed in our privacy policy. Sending homeowner details is the same category, so this is a confirmation rather than a blocker — but it should be a decision, not a side effect.

Estimates are the engineering half only, and assume no discovery surprises in the Go High Level API — we have not used it before, only its webhooks. Companion document: the audit of every automated email the product sends today.